Project 4: Password Strength Analyser & Custom Wordlist Generator

Name: Mohammed Fadil PK

Domain: Cybersecurity / Offensive Security (Red Team)

1. Introduction

In offensive security, understanding how passwords are created and cracked is essential. Attackers rarely depend on purely random guesses; instead, they use Open Source Intelligence (OSINT) to create highly targeted password lists. This project focuses on developing a Python toolkit that measures password entropy and algorithmically generates custom wordlists for dictionary attacks.

2. Abstract

The "Red Team Password Toolkit" is a dual-function Command Line Interface (CLI) application. The first module is a Strength Analyser that uses the industry-standard zxcvbn engine to calculate password entropy and estimate offline cracking time. The second module is a Custom Wordlist Generator that takes specific target data (names, dates, etc.) and uses permutation logic and "leetspeak" substitutions to generate a customized.txt dictionary file.

3. Tools & Technologies Used

- **Operating System:** Kali Linux.
- **Language:** Python 3.
- **Libraries:** zxcvbn (for entropy calculation), getpass (for secure input masking), itertools (for algorithmic permutations).

4. Steps Involved in Building the Project

Step 1: Environment Setup & Engine Integration

- Configured the Python environment and installed the open-source zxcvbn password strength estimation library to handle advanced entropy mathematics.

Step 2: Developing the Strength Analyser Logic

- Built the check_password_strength() module, incorporating getpass to hide user input in the Linux terminal securely.
- Programmed the tool to extract the security score (0-4), estimate offline cracking time, and display specific vulnerability feedback (e.g., identifying common patterns).

Step 3: Implementing the Wordlist Generator Logic

- Developed the generate_wordlist() module to accept targeted OSINT inputs (Name, Pet Name, Year).
- Utilised itertools.permutations to generate all possible combinations of the provided base words.
- Implemented a character-mapping loop to dynamically generate "leetspeak" variations (e.g., replacing 'a' with '@', 'o' with '0') to mimic common human password behaviours.

Step 4: Data Export

- Programmed the script to remove duplicate entries, sort the final attack dictionary, and export it as a clean custom_wordlist.txt file suitable for brute-force tools.

5. Results (Evidence)

Screenshot 1: Password Entropy Analysis

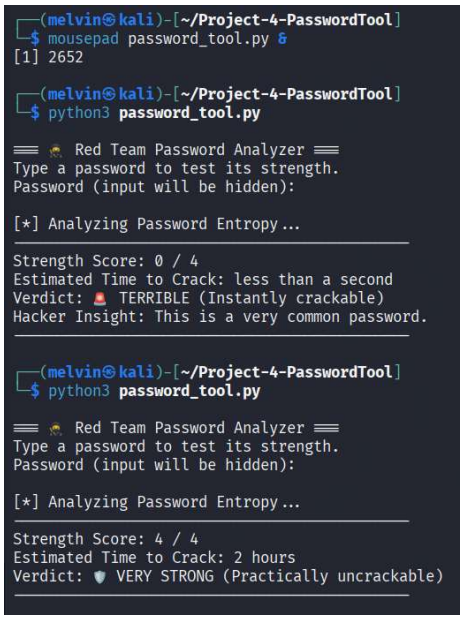

Figure 1: *Execution of the Strength Analyser evaluating a weak and a strong password, accurately estimating crack times.*

Screenshot 2: Generated Custom Wordlist

```
┌──(melvin㉿kali)-[~/Project-4-PasswordTool]
└─$ cat custom_wordlist.txt
2000
2000@br@h@m3223
2000abraham3223
2000m0h@mm@d
2000mohammad
@br@h@m3223
@br@h@m32232000
@br@h@m3223m0h@mm@d
Abr@h@m3223
Abr@h@m32232000
Abr@h@m3223m0h@mm@d
Abraham3223
Abraham32232000
Abraham3223mohammad
M0h@mm@d
M0h@mm@d2000
M0h@mm@d@br@h@m3223
Mohammad
Mohammad2000
Mohammadabraham3223
abraham3223
abraham32232000
abraham3223mohammad
m0h@mm@d
m0h@mm@d2000
m0h@mm@d@br@h@m3223
mohammad
mohammad2000
mohammadabraham3223
```

Figure 2: *The output of cat custom_wordlist.txt, demonstrating successful algorithmic permutations and leetspeak generation.*

6. Conclusion

This project successfully demonstrates the mechanisms behind dictionary attacks and password entropy. By building the tools that attackers use to evaluate and crack credentials, security professionals can better educate users on creating resilient passwords and implement stricter password policies at the organisational level.